TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

LẬP TRÌNH THI ĐẤU

SỐ HỌC

NỀN TẢNG

BẮT ĐẦU TỪ 0
không cần nền tảng

Dành cho học sinh **mới bắt đầu** · Ôn thi **HSG cấp Tỉnh** · THCS & THPT



Lê Thành Nam

`n % k` `gcd(a, b)` `aⁿ mod m` `√n` `p | ab ⇒ p | a ∨ p | b`

`lcm = a / g · b` `(1101)₂ = 13` `C(n, k)`

01	02	03	04	05	06
Phép chia và đồng dư	Chữ số và hệ cơ số	Số nguyên tố	Ước và bội	Lũy thừa và dãy số	Số lớn và tổ hợp

TẬP 1 · LÝ THUYẾT

16 bài học · 6 chương
lý thuyết đầy đủ trước, bài toán mẫu sau

Chia hết · Đồng dư · Sàng · Euclid · Lũy thừa nhanh · Số lớn

Tổng hợp và biên soạn: **Lê Thành Nam**

LỜI NÓI ĐẦU

Gần như đề thi học sinh giỏi Tin học cấp tỉnh nào cũng có ít nhất một bài **số học**: Đếm số chia hết cho k trong một đoạn, kiểm tra số nguyên tố, tìm ước chung lớn nhất, tính lũy thừa theo môđun, cộng hai số có hàng nghìn chữ số. Những bài này thường được xem là "dễ", vì ý tưởng có thể chỉ gói trong một dòng công thức. Thực tế phòng thi cho thấy điều ngược lại: Đây là nơi học sinh mất điểm nhiều nhất một cách **không ngờ**. Một phép chia số âm, một phép nhân tràn số, một lần gọi sqrt, một phép "lấy dư rồi mới chia": Chương trình vẫn chạy rất nhanh, in ra những con số trông rất hợp lí, và sai.

Cuốn sách này được viết để em làm chủ phần số học ở mức cấp tỉnh **từ con số 0**: Chỉ cần biết vòng lặp, mảng, hàm, và hiểu thế nào là độ phức tạp thuật toán. Mỗi kiến thức đều được trình bày theo cùng một trình tự: định nghĩa chính xác, định lí kèm chứng minh, ví dụ chạy tay, rồi mới đến chương trình. Không có công thức nào "cứ thế mà nhớ".

Bộ tài liệu gồm 3 đi cùng nhau:

• **Tập 1: Lý thuyết** (cuốn sách này) gồm 16 bài học chia thành 6 chương: Phép chia và đồng dư; Chữ số và hệ cơ số; Số nguyên tố; Ước và bội; Lũy thừa và dãy số; Số lớn và tổ hợp. Mỗi bài học trình bày **toàn bộ lý thuyết trước**, sau đó là **hai bài toán mẫu giải chi tiết**, mỗi bài đi từ cách ngây thơ nhất tới cách tối ưu, qua ít nhất ba bước.

• **Tập 2: Bài tập** gồm 48 bài, mỗi bài học có ba bài tập ở ba mức độ, đề viết theo đúng thể thức đề thi với ràng buộc chia theo subtask.

• **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ tới cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thật của từng cách làm trên bộ test, kể cả những cách sai.

Một điều đặc biệt của cuốn sách: **Mọi con số đều là số đo thật**. Mỗi chương trình trong sách đã được biên dịch và chạy; thời gian chạy ghi trong các khung "Đếm số phép tính" là thời gian đo được trên máy; mọi cách làm của cùng một bài toán đã được đối chiếu với nhau trên hàng trăm bộ dữ liệu ngẫu nhiên; khi sách nói một cách làm "sai $\frac{272}{300}$ bộ thử", đó là kết quả chạy thật. Những cách làm sai được giữ lại có chủ đích: Nhìn thấy một chương trình sai ở đâu và sai bao nhiêu là cách nhanh nhất để không bao giờ mắc lỗi đó.

Chương trình được viết bằng C++ theo chuẩn C++14 (trừ kiểu `__int128` là phần mở rộng của trình biên dịch g++), đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Chương trình không có chú thích bên trong; mọi giải thích được viết thành lời ngay sau mỗi chương trình.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học gồm hai phần lớn:

Phần	Nội dung
Mục tiêu, Khởi động	Những gì em sẽ làm được sau bài học, và một câu hỏi dẫn tới ý tưởng chính.
Phần Lý Thuyết	Toàn bộ kiến thức của bài: định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình minh hoạ. Đọc phần này với giấy bút và tự chạy lại mọi ví dụ.
Phần Bài toán	Hai bài toán lập trình dùng đúng lý thuyết vừa học. Mỗi bài đi qua các cách làm từ ngây thơ tới tối ưu; mỗi cách có chương trình hoàn chỉnh, giải thích, độ phức tạp và thời gian chạy đo thật.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Luyện tập	Liên hệ tới ba bài tập ở Tập 2 và ba câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
Khởi động	Câu hỏi mở đầu bài học.
Định nghĩa	Khái niệm mới, cần hiểu chính xác.
Định lí	Kết quả quan trọng, luôn kèm chứng minh.
Chứng minh	Lập luận vì sao định lí đúng.
Ví dụ	Chạy tay trên số cụ thể.
Công thức	Công thức cần nhớ.
Bài toán	Đề bài lập trình của bài học.
Đếm số phép tính	Độ phức tạp và số đo thời gian thật của từng cách làm.
Vì sao đúng?	Giải thích thêm một bước khó hoặc một dòng lệnh quan trọng.
Lưu ý	Chi tiết dễ bỏ sót.
Bẫy thường gặp	Các lỗi hay gặp.
Ghi nhớ	Tóm tắt cuối phần lý thuyết.

Lê Thành Nam

Kí hiệu. $a \mid b$ đọc là "a chia hết b" (a là ước của b). $a \bmod m$ là số dư **không âm** của phép chia a cho m; còn $a \,\%\, m$ là phép toán của C++, có thể âm. $\lfloor x \rfloor$, $\lceil x \rceil$ là làm tròn xuống, làm tròn lên. gcd, lcm là ước chung lớn nhất, bội chung nhỏ nhất. $log\ n$ luôn hiểu là $log_2\ n$. Trong các chương trình, ll là viết tắt của $long\ long$ (khai báo bằng $typedef\ long\ long\ ll;$).

Thời gian chạy. Các số đo trong sách là thời gian CPU trên một máy tính thông thường, biên dịch với $g++ -O2 -std=c++14$, lấy giá trị nhỏ nhất của vài lần chạy. Máy chấm của em có thể nhanh hơn hoặc chậm hơn vài lần, nhưng **tỉ lệ** giữa các cách làm thì không đổi: Một thuật toán $O(n)$ chậm hơn thuật toán $O(log\ n)$ hàng triệu lần ở bất kì máy nào. Khi sách ghi "quá 8 giây", chương trình đã bị dừng sau 8 giây mà chưa xong.

Một quy tắc ước lượng. Một giây, máy tính thực hiện được khoảng $10^8 - 10^9$ phép tính đơn giản. Phép chia và phép lấy dư chậm hơn phép cộng, phép nhân vài lần. Trước khi viết chương trình, hãy nhân thử: Số câu hỏi $\times$ số phép tính mỗi câu hỏi.

Lời khuyên khi học. Với mỗi định lí, hãy tự chứng minh lại sau khi gấp sách. Với mỗi chương trình, hãy tự hỏi: Giá trị lớn nhất mà mỗi biến có thể nhận là bao nhiêu? Có số âm không? Có phép chia nào không chính xác không? Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. PHÉP CHIA VÀ ĐỒNG DƯ **6**

Bài 1. Phép chia có dư 6
Bài 2. Đồng dư thức và phép toán modulo 18
Bài 3. Chu kì của số dư và chữ số tận cùng 28

CHƯƠNG 2. CHỮ SỐ VÀ HỆ CƠ SỐ **38**

Bài 4. Chữ số của một số 38
Bài 5. Hệ cơ số 49

CHƯƠNG 3. SỐ NGUYÊN TỐ **60**

Bài 6. Kiểm tra số nguyên tố 60
Bài 7. Sàng Eratosthenes 71
Bài 8. Phân tích thừa số nguyên tố 80

CHƯƠNG 4. ƯỚC VÀ BỘI **92**

Bài 9. Ước số 92
Bài 10. Ước chung lớn nhất và thuật toán Euclid 102
Bài 11. Bội chung nhỏ nhất và phân số 111

CHƯƠNG 5. LŨY THỪA VÀ DÃY SỐ **122**

Bài 12. Lũy thừa nhanh 122
Bài 13. Số chính phương và căn bậc hai nguyên 132
Bài 14. Dãy số và công thức tổng 141

CHƯƠNG 6. SỐ LỚN VÀ TỔ HỢP **151**

Bài 15. Tràn số và số lớn 151
Bài 16. Tổ hợp cơ bản và tam giác Pascal 161

PHỤ LỤC: BẢNG TRA CỨU NHANH **172**

Lê Thành Nam

CHƯƠNG 1

PHÉP CHIA VÀ ĐỒNG DƯ

*Số học trong lập trình thi đấu bắt đầu từ một phép toán em đã biết từ tiểu học: **Phép chia có dư**. Ba bài học trong chương này giúp em nắm vững phép chia đó theo đúng cách mà máy tính thực hiện nó. Bài 1 trình bày **định lí chia có dư** và sự khác nhau giữa phép chia trong toán học và phép chia của C++ khi gặp số âm. Bài 2 xây dựng **đồng dư thức**, công cụ cho phép làm việc với những con số khổng lồ mà chỉ cần giữ lại phần dư. Bài 3 khai thác **tính tuần hoàn của số dư** để trả lời những câu hỏi như "chữ số tận cùng của $7^{10^{18}}$ là bao nhiêu" chỉ trong vài phép tính.*

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Phép chia có dư
- **Bài 2.** Đồng dư thức và phép toán modulo
- **Bài 3.** Chu kì của số dư và chữ số tận cùng

BÀI 1 Phép chia có dư

Học xong bài này, em sẽ:

✓ Phát biểu và chứng minh được **định lí chia có dư**; hiểu chính xác thương và số dư khi số bị chia là số âm.

✓ Biết phép chia `/` và phép lấy dư `%` của C++ làm tròn theo hướng nào, và viết được hàm chia làm tròn xuống, làm tròn lên, lấy dư không âm.

✓ Đếm số bội của k trong một đoạn [a, b] bất kì chỉ bằng **một công thức**, kể cả khi a, b âm và lớn tới 10^{18}.

✓ Dùng cặp phép toán chia lấy nguyên - chia lấy dư để đổi đơn vị, đánh số ô trong bảng và giải các bài toán "xếp hàng, xếp phòng".

Lê Thành Nam

■ KHỞI ĐỘNG

Có bao nhiêu số nguyên trong đoạn từ -10^{18} tới 10^{18} chia hết cho 3?

Một chương trình duyệt từng số cần $2 \cdot 10^{18}$ lần kiểm tra. Máy tính thực hiện khoảng 10^9 phép tính mỗi giây, nên chương trình đó chạy **hơn 60 năm**.

Sau bài học này, em sẽ trả lời câu hỏi trên bằng một phép trừ của hai phép chia, và chương trình chạy xong trong vài phần nghìn giây.

Nhưng có một cái bẫy: Nếu viết công thức đó bằng phép chia `/` của C++ theo cách tự nhiên nhất, đáp số sẽ sai đúng một đơn vị.

PHẦN LÝ THUYẾT

1. Chia hết, ước và bội

■ CHIA HẾT

Cho hai số nguyên a và b với $b \neq 0$.

Ta nói **a chia hết cho b** nếu tồn tại số nguyên q sao cho $a = b \cdot q$. Khi đó ta viết **b | a** (đọc là "b chia hết a" hoặc "b là ước của a"), và nói a là **bội** của b.

Nếu a không chia hết cho b, ta viết $b \nmid a$.

Vài trường hợp đặc biệt cần nhớ ngay:

- Số 0 chia hết cho mọi số $a \neq 0$, vì $0 = b \cdot 0$. Vậy 0 là bội của mọi số.
- Các số 1 và -1 là ước của mọi số nguyên, vì $a = 1 \cdot a = (-1) \cdot (-a)$.
- Nếu b | a thì -b | a, b | -a và -b | -a. Khi nói về ước, người ta thường chỉ xét các **ước dương**.

■ TÍNH CHẤT CỦA QUAN HỆ CHIA HẾT

Với các số nguyên a, b, c (các số chia khác 0):

- **(T1) Bắc cầu:** nếu c | b và b | a thì c | a.
- **(T2) Tổ hợp:** nếu d | a và d | b thì d | (x·a + y·b) với mọi số nguyên x, y.
- Đặc biệt d | (a + b) và d | (a - b).
- **(T3) So sánh:** nếu b | a và $a \neq 0$ thì $|b| \leq |a|$.

■ CHỨNG MINH

- (T1): $b = c \cdot u$ và $a = b \cdot v$ với u, v nguyên, nên $a = c \cdot (u \cdot v)$, mà $u \cdot v$ là số nguyên.
- (T2): $a = d \cdot u$ và $b = d \cdot v$, nên $x \cdot a + y \cdot b = d \cdot (x \cdot u + y \cdot v)$.
- (T3): $a = b \cdot q$ với q nguyên; vì $a \neq 0$ nên $q \neq 0$, tức $|q| \geq 1$, do đó $|a| = |b| \cdot |q| \geq |b|$.

Lê Thanh Nam

■ VÍ DỤ

7 | 91 vì 91 = 7·13. Ta cũng có 7 | 14. Theo (T2), 7 chia hết 91 - 6·14 = 7, điều này hiển nhiên đúng. Ngược lại, (T2) cho phép kết luận nhanh rằng 7 ∤ 93: nếu 7 | 93 thì 7 | 93 - 91 = 2, trái với (T3) vì 7 > 2.

Tính chất (T2) xuất hiện trong gần như mọi chứng minh của cuốn sách. Em sẽ gặp lại nó trong thuật toán Euclid (Bài 10) và trong sàng Eratosthenes (Bài 7).

2. Định lí chia có dư

Với hai số nguyên a, b bất kì (b > 0), a không nhất thiết chia hết cho b. Định lí sau nói rằng ta luôn chia được a cho b với một **phần dư** nhỏ hơn b, và cách chia đó là duy nhất.

■ ĐỊNH LÍ CHIA CÓ DƯ

Cho số nguyên a và số nguyên dương b. Tồn tại **duy nhất** một cặp số nguyên (q, r) sao cho

$$a = b\cdot q + r \quad \text{và} \quad 0 \le r < b.$$

Số q gọi là **thương**, số r gọi là **số dư** của phép chia a cho b.

Để chứng minh, ta cần khái niệm phần nguyên.

■ PHẦN NGUYÊN (LÀM TRÒN XUỐNG) VÀ LÀM TRÒN LÊN

Với số thực x, **⌊x⌋** là số nguyên lớn nhất không vượt quá x, và **⌈x⌉** là số nguyên nhỏ nhất không nhỏ hơn x. Theo định nghĩa, $\lfloor x\rfloor \le x < \lfloor x\rfloor + 1$ và $\lceil x\rceil - 1 < x \le \lceil x\rceil$.

Ví dụ: ⌊3,4⌋ = 3, ⌈3,4⌉ = 4, ⌊-3,4⌋ = -4 (không phải -3, vì -3 > -3,4), ⌈-3,4⌉ = -3, ⌊5⌋ = ⌈5⌉ = 5.

■ CHỨNG MINH

Tồn tại. Đặt q = ⌊a/b⌋. Theo định nghĩa phần nguyên, q ≤ a/b < q + 1. Nhân cả ba vế với b > 0: b·q ≤ a < b·q + b. Đặt r = a - b·q, ta có 0 ≤ r < b và a = b·q + r.

Duy nhất. Giả sử a = b·q + r = b·q' + r' với 0 ≤ r, r' < b. Trừ hai đẳng thức: b·(q - q') = r' - r. Vế phải thoả mãn -b < r' - r < b. Vế trái là một bội của b. Bội duy nhất của b nằm hẳn trong khoảng (-b, b) là 0, nên r' = r, kéo theo q = q'.

Chứng minh trên cho thêm một thông tin quan trọng: **Thương luôn là q = ⌊a/b⌋**, tức là a/b **làm tròn xuống**, kể cả khi a âm.

Lê Thành Nam

■ VÍ DỤ

- a = 17, b = 5: 17 = 5·3 + 2. Thương 3, dư 2.
- a = -17, b = 5: -17 = 5·(-4) + 3. Thương -4, dư 3. Cách viết -17 = 5·(-3) + (-2) **không** phải phép chia có dư vì số dư -2 âm.
- a = 0, b = 5: 0 = 5·0 + 0.
- a = 4, b = 5: 4 = 5·0 + 4. Khi $0 \le a < b$, thương bằng 0 và số dư chính là a.

■ HỆ QUẢ

Với b > 0: **b | a khi và chỉ khi số dư của phép chia a cho b bằng 0.** Ngoài ra, a và số dư r có cùng số dư khi chia cho b, và b | (a - r).

3. Phép chia của C++

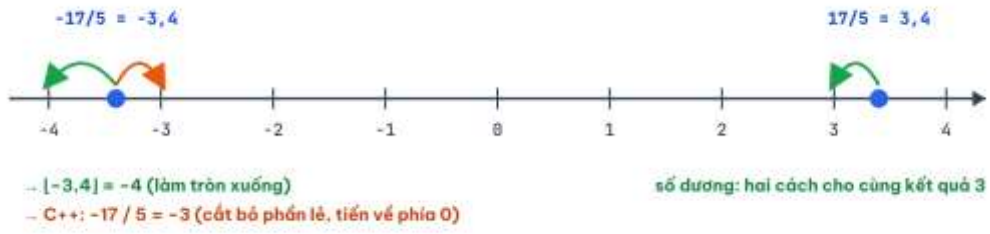
Trong C++ có hai phép toán cho số nguyên: `a / b` (chia lấy phần nguyên) và `a % b` (chia lấy dư). Từ chuẩn C++11 trở đi, quy tắc được cố định như sau.

■ QUY TẮC CỦA C++ (b ≠ 0)

- `a / b` là thương a/b **cắt bỏ phần thập phân**, tức là làm tròn về phía 0.
- `a % b` được định nghĩa sao cho luôn có **(a / b)·b + a % b = a**. Hệ quả: `a % b` có **cùng dấu với a** (hoặc bằng 0) và |a % b| < |b|.

Với $a \ge 0$ và b > 0, "cắt bỏ phần thập phân" và "làm tròn xuống" là một, nên `a / b` = ⌊a/b⌋ và `a % b` đúng là số dư của định nghĩa chia có dư. Khác biệt chỉ xuất hiện khi **a và b trái dấu** và a không chia hết cho b. Bảng dưới đây em nên tự kiểm tra bằng một chương trình nhỏ.

a	b	a / b	a % b	⌊a/b⌋ (toán)	số dư (toán)
17	5	3	2	3	2
-17	5	**-3**	**-2**	-4	3
17	-5	-3	2	-4	(không xét)
-17	-5	3	-2	3	(không xét)
-15	5	-3	0	-3	0



Hình 1.1. Thương của C++ và phần nguyên toán học khác nhau ở số âm

Lê Thanh Nam

Trong lập trình thi đấu, số chia hầu như luôn dương. Vì vậy ta chỉ cần ba hàm sau, với điều kiện b > 0.

‹› Ba hàm chia an toàn với số âm (b > 0)

```
ll floorDiv(ll a, ll b) {
    ll q = a / b;
    if (a % b != 0 && a < 0) q--;
    return q;
}
ll ceilDiv(ll a, ll b) {
    ll q = a / b;
    if (a % b != 0 && a > 0) q++;
    return q;
}
ll modPos(ll a, ll b) {
    return (a % b + b) % b;
}
```

■ VÌ SAO ĐÚNG?

- **floorDiv:** nếu b | a, phép chia chính xác nên `a / b` đúng. Nếu không chia hết và a > 0, cắt bỏ cũng là làm tròn xuống. Chỉ khi a < 0 thì cắt bỏ phần thập phân làm kết quả **lớn hơn** ⌊a/b⌋ đúng 1 đơn vị (-3 thay vì -4), nên ta trừ đi 1.
- **ceilDiv:** đối xứng với floorDiv. Khi a > 0 và không chia hết, cắt bỏ làm kết quả **nhỏ hơn** ⌈a/b⌉ đúng 1, nên cộng thêm 1.
- **modPos:** `a % b` nằm trong khoảng (-b, b). Cộng b được một số trong khoảng (0, 2b) có cùng số dư với a khi chia cho b. Lấy `% b` lần nữa được số trong [0, b). Với a = -17, b = 5: -17 % 5 = -2, -2 + 5 = 3, 3 % 5 = 3. Đúng là số dư toán học.

Có một công thức làm tròn lên rất hay gặp: với a ≥ 0 và b > 0,

■ LÀM TRÒN LÊN KHI a ≥ 0

⌈a/b⌉ = (a + b - 1) / b (phép chia của C++).

■ CHỨNG MINH

Viết a = b·q + r với 0 ≤ r < b. Nếu r = 0: a + b - 1 = b·q + (b - 1), mà 0 ≤ b - 1 < b, nên thương là q = ⌈a/b⌉. Nếu r ≥ 1: a + b - 1 = b·(q + 1) + (r - 1), mà 0 ≤ r - 1 < b, nên thương là q + 1. Lúc này a/b = q + r/b với 0 < r/b < 1, nên ⌈a/b⌉ = q + 1.

■ LƯU Ý

Công thức (a + b - 1)/b gọn nhưng có hai nhược điểm. Thứ nhất, nó **sai khi a < 0**. Thứ hai, a + b - 1 có thể **vượt quá phạm vi long long** (khoảng $9{,}22 \cdot 10^{18}$) dù cả a và b đều nằm trong phạm vi. Cách viết `a / b + (a % b != 0)` không có hai nhược điểm đó khi a ≥ 0. Phần bài toán 2 sẽ cho thấy lỗi tràn số này thật sự làm sai đáp số.

Lê Thành Nam

4. Đếm bội trong một đoạn

Đây là ứng dụng quan trọng nhất của phần nguyên trong bài học này.

■ SỐ BỘI CỦA k KHÔNG VƯỢT QUÁ n

Với số nguyên $n \geq 0$ và số nguyên dương k, số các số nguyên dương không vượt quá n và chia hết cho k là $\lfloor n/k \rfloor$.

■ CHỨNG MINH

Các bội dương của k là k, 2k, 3k, ... Bội $m \cdot k$ ($m \geq 1$) không vượt quá n khi và chỉ khi $m \leq n/k$, khi và chỉ khi $m \leq \lfloor n/k \rfloor$ (vì m nguyên). Vậy các giá trị hợp lệ của m là 1, 2, ..., $\lfloor n/k \rfloor$.

Ví dụ: từ 1 tới 100 có $\lfloor 100/7 \rfloor = 14$ số chia hết cho 7 (từ 7 tới 98). Mở rộng ra một đoạn bất kì:

■ SỐ BỘI CỦA k TRONG ĐOẠN [a, b]

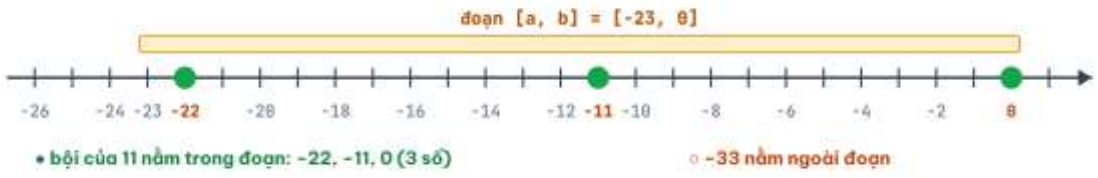
Với mọi số nguyên $a \leq b$ (có thể âm) và số nguyên dương k, số các số nguyên x thoả mãn $a \leq x \leq b$ và $k \mid x$ là

$$\lfloor b/k \rfloor - \lfloor (a - 1)/k \rfloor.$$

■ CHỨNG MINH

x là bội của k khi $x = k \cdot m$ với m nguyên. Điều kiện $a \leq k \cdot m \leq b$ tương đương $a/k \leq m \leq b/k$, tức là $\lceil a/k \rceil \leq m \leq \lfloor b/k \rfloor$. Số giá trị nguyên của m là $\lfloor b/k \rfloor - \lceil a/k \rceil + 1$ (bằng 0 nếu không có giá trị nào, vì khi đó $\lceil a/k \rceil = \lfloor b/k \rfloor + 1$).

Còn lại phải chứng minh $\lceil a/k \rceil = \lfloor (a - 1)/k \rfloor + 1$ với mọi số nguyên a. Chia có dư: $a - 1 = k \cdot q + r$ với $0 \leq r < k$, vậy $\lfloor (a - 1)/k \rfloor = q$. Khi đó $a = k \cdot q + (r + 1)$ với $1 \leq r + 1 \leq k$, suy ra $q < a/k \leq q + 1$, tức $\lceil a/k \rceil = q + 1$.



Hình 1.2. Bội của 11 trong đoạn [-23, 0]

■ VÍ DỤ

$a = -23$, $b = 0$, $k = 11$. Công thức cho $\lfloor 0/11 \rfloor - \lfloor -24/11 \rfloor = 0 - (-3) = 3$, đúng với ba số -22, -11, 0 trên Hình 1.2. Nếu tính $\lfloor -24/11 \rfloor$ bằng phép chia `/` của C++, ta được `-24 / 11 = -2` (cắt bỏ phần thập phân của -2,18...), và đáp số sai thành $0 - (-2) = 2$.

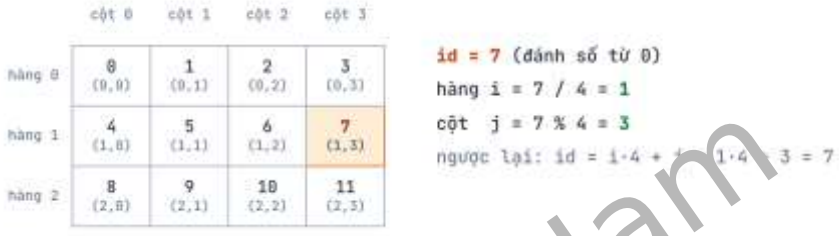
Lê Thành Nam

5. Chia lấy nguyên và chia lấy dư để đổi đơn vị, đánh số

Định lí chia có dư nói rằng mỗi số tự nhiên a được "tách" duy nhất thành cặp (thương, dư) khi chia cho b. Cách tách này có mặt ở khắp nơi trong lập trình.

Đổi đơn vị. t giây gồm t / 86400 ngày; phần còn lại t % 86400 giây gồm (t % 86400) / 3600 giờ; tiếp tục như vậy với phút và giây. Ví dụ t = 100000: 100000 = 86400·1 + 13600; 13600 = 3600·3 + 2800; 2800 = 60·46 + 40. Vậy 100000 giây là 1 ngày 3 giờ 46 phút 40 giây.

Đánh số ô trong bảng. Một bảng có R hàng, m cột. Nếu đánh số các ô từ 0 theo từng hàng, từ trái qua phải, thì ô số id nằm ở hàng i = id / m và cột j = id % m (hàng, cột cũng đánh số từ 0). Ngược lại, ô (i, j) có số id = i·m + j. Đây chính là định lí chia có dư: id = m·i + j với 0 ≤ j < m.



Hình 1.3. Bảng 3 hàng, 4 cột đánh số từ 0

Khi đề bài đánh số **từ 1**, hãy đổi về đánh số từ 0 trước (trừ 1), dùng công thức, rồi cộng 1 vào kết quả: ô số id (từ 1) ở hàng (id - 1)/m + 1 và cột (id - 1) % m + 1. Quên bước trừ 1 là một lỗi rất phổ biến: với m = 4, ô số 4 (từ 1) phải ở hàng 1 cột 4, nhưng 4 / 4 + 1 = 2 lại cho hàng 2.

Lặp vòng tròn. Nếu hôm nay là thứ d (đánh số 0 là Chủ nhật, 1 là thứ Hai, ..., 6 là thứ Bảy) thì n ngày sau là thứ (d + n) mod 7. Khi n âm (n ngày trước), phải dùng modPos thay vì %.

■ GHI NHỚ

- a = b·q + r, 0 ≤ r < b, với **q = ⌊a/b⌋**: thương toán học luôn làm tròn xuống.
- C++ làm tròn **về phía 0**; `a % b` cùng dấu với a. Hai cách chỉ khác nhau khi a âm (với b > 0).
- Số bội của k trong [a, b] là ⌊b/k⌋ - ⌊(a - 1)/k⌋, với phép chia làm tròn xuống thật sự.
- ⌈a/b⌉ = a / b + (a % b != 0) khi a ≥ 0; công thức (a + b - 1)/b có thể tràn số.
- Đổi đánh số từ 1 sang đánh số từ 0 trước khi dùng / và %.

Lê Thành Nam

PHẦN BÀI TOÁN

■ BÀI TOÁN 1: DEMBOI

Có q câu hỏi. Câu hỏi thứ j cho ba số nguyên a, b, k; hãy đếm số các số nguyên x với $a \le x \le b$ và x chia hết cho k.

Dữ liệu vào: Dòng đầu gồm số q ($1 \le q \le 10^5$). Mỗi dòng trong q dòng tiếp theo gồm ba số a, b, k ($-10^{18} \le a \le b \le 10^{18}$; $1 \le k \le 10^{18}$).

Kết quả: Mỗi câu hỏi in ra đáp số trên một dòng.

DEMBOI.INP	DEMBOI.OUT	Giải thích
3 1 10 3 -23 0 11 -5 -1 7	3 3 0	3, 6, 9 chia hết cho 3. -22, -11, 0 (Hình 1.2). Không có bội nào của 7 trong [-5, -1].

6. Cách 1 - Duyệt từng số

Cách tự nhiên nhất: thử từng số x từ a tới b, đếm các số có `x % k == 0`. Điều kiện `x % k == 0` đúng cả với x âm, vì `x % k` chỉ bằng 0 khi k | x (từ quy tắc (a / b)·b + a % b = a).

Cách 1: duyệt từng số, O(b - a)

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int q;
    cin >> q;
    while (q--) {
        ll a, b, k;
        cin >> a >> b >> k;
        ll cnt = 0;
        for (ll x = a; x <= b; x++)
            if (x % k == 0) cnt++;
        cout << cnt << '\n';
    }
    return 0;
}
```

Chương trình đúng nhưng mỗi câu hỏi tốn b - a + 1 phép kiểm tra. Đoạn $[1, 10^9]$ đã mất **3,30 giây**; đoạn $[-10^{18}, 10^{18}]$ không bao giờ chạy xong.

7. Cách 2 - Nhảy theo bước k

Quan sát: các bội của k cách nhau đúng k đơn vị. Nếu đã biết bội đầu tiên $x \ge a$ thì các bội tiếp theo là x + k, x + 2k, ... Ta tìm bội đầu tiên bằng cách tăng x từ a cho tới khi `x % k == 0` (tối đa k - 1 bước), rồi nhảy từng bước k.

Lê Thành Nam

Cách 2: tìm bội đầu tiên rồi nhảy bước k

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int q;
    cin >> q;
    while (q--) {
        ll a, b, k;
        cin >> a >> b >> k;
        ll x = a;
        while (x <= b && x % k != 0) x++;
        ll cnt = 0;
        while (x <= b) {
            cnt++;
            if (x > b - k) break;
            x += k;
        }
        cout << cnt << '\n';
    }
    return 0;
}
```

Dòng 16 kiểm tra `x > b - k` trước khi cộng: nếu cộng thẳng `x += k` với x gần 10^{18} và k gần 10^{18}, tổng có thể vượt phạm vi long long. Viết `x > b - k` thay cho `x + k > b` là kĩ thuật tránh tràn số em sẽ dùng nhiều lần: chuyển phép cộng có thể tràn sang vế bên kia thành phép trừ không tràn.

Cách 2 tốn khoảng k + (b - a)/k bước. Với đoạn [1, 10^9] và k = 7 chỉ còn **0,047 giây** (khoảng $1{,}4 \cdot 10^8$ bước nhảy). Nhưng nó vẫn chậm ở hai tình huống: k rất lớn (bước tìm bội đầu tiên dài, k = 10^8 mất **0,32 giây** cho một câu hỏi) và k rất nhỏ trên đoạn rất dài (k = 3 trên đoạn dài $2 \cdot 10^{18}$ cần gần $7 \cdot 10^{17}$ bước nhảy).

8. Cách 3 - Công thức phần nguyên

Áp dụng định lí ở mục 4: đáp số là ⌊b/k⌋ - ⌊(a - 1)/k⌋. Nhiều học sinh viết ngay:

Bản lỗi: dùng thẳng phép chia của C++

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int q;
    cin >> q;
    while (q--) {
        ll a, b, k;
        cin >> a >> b >> k;
        cout << b / k - (a - 1) / k << '\n';
    }
    return 0;
}
```

Chương trình này đúng ở ví dụ thứ nhất và thứ ba, nhưng ví dụ thứ hai in ra **2** thay vì 3. Lý do đã phân tích ở mục 4: (-24) / 11 trong C++ bằng -2, còn ⌊-24/11⌋ = -3. Ở ví dụ thứ ba, (-6) / 7 = 0 và (-1) / 7 = 0 nên hiệu vẫn đúng bằng 0 một cách tình cờ. Chỉ cần thay phép chia bằng hàm floorDiv của mục 3:

Lê Thành Nam

DEMBOI.CPP: công thức phần nguyên, O(1) mỗi câu hỏi

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  ll floorDiv(ll a, ll b) {
6      ll q = a / b;
7      if (a % b != 0 && a < 0) q--;
8      return q;
9  }
10
11 int main() {
12     ios_base::sync_with_stdio(false);
13     cin.tie(nullptr);
14     if (fopen("DEMBOI.INP", "r")) {
15         freopen("DEMBOI.INP", "r", stdin);
16         freopen("DEMBOI.OUT", "w", stdout);
17     }
18     int q;
19     cin >> q;
20     while (q--) {
21         ll a, b, k;
22         cin >> a >> b >> k;
23         cout << floorDiv(b, k) - floorDiv(a - 1, k) << '\n';
24     }
25     return 0;
26 }
```

Hàm floorDiv (dòng 5 - 9) giảm thương đi 1 khi [illegible] âm và không chia hết cho b. Dòng 23 áp dụng đúng công thức của định lí. Không có phép tính nào tràn số: a - 1 ≥ -10^{18} - 1, vẫn nằm trong phạm vi long long. Mỗi câu hỏi chỉ tốn vài phép tính.

■ ĐẾM SỐ PHÉP TÍNH

- Cách 1: O(b - a) mỗi câu hỏi. Đoạn [1, 10^9]: **3,30 giây**. Đoạn [-10^{18}, 10^{18}]: không kết thúc (quá 6 giây khi dừng thử; ước tính hơn 60 năm).
- Cách 2: O(k + (b - a)/k). Đoạn [1, 10^9], k = 7: **0,047 giây**; k = 10^8: **0,32 giây**. Đoạn [-10^{18}, 10^{18}], k = 3: không kết thúc.
- Cách 3: O(1) mỗi câu hỏi. Mọi trường hợp trên: **0,003 - 0,004 giây**; 10^5 câu hỏi ngẫu nhiên với số tới 10^{18}: **0,036 giây**, gần như toàn bộ là thời gian đọc dữ liệu.

Ba chương trình đã được đối chiếu với nhau trên 500 bộ dữ liệu ngẫu nhiên nhỏ có cả số âm. Bản lỗi sai ngay ở bộ thứ nhất.

Lê Thành Nam

■ BÀI TOÁN 2: XEPPHONG

Trường tổ chức thi cho n học sinh, mỗi phòng thi xếp tối đa k em. Học sinh được xếp lần lượt theo số báo danh 1, 2, ..., n: k em đầu vào phòng 1, k em tiếp theo vào phòng 2, ... Có q câu hỏi, mỗi câu cho n, k, i. Hãy cho biết cần ít nhất bao nhiêu phòng, học sinh số báo danh i ngồi ở phòng nào và ở chỗ số mấy trong phòng đó (chỗ đánh số từ 1).

Dữ liệu vào: Dòng đầu gồm q ($1 \le q \le 10^5$). Mỗi dòng tiếp theo gồm n, k, i ($1 \le n, k \le 9 \cdot 10^{18}$; $1 \le i \le n$).

Kết quả: Mỗi câu hỏi in ba số: số phòng, phòng của học sinh i, chỗ của học sinh i.

XEPPHONG.INP	XEPPHONG.OUT	Giải thích
2 10 4 7 900000000000000000 900000000000000000 1	3 2 3 1 1 1	Phòng 1: em 1 - 4; phòng 2: em 5 - 8 (em 7 ở chỗ 3); phòng 3: em 9, 10. Một phòng chứa được tất cả.

9. Cách 1 - Mô phỏng

Lấy dần k em một cho tới khi hết; với học sinh i, trừ dần k cho tới khi i còn không quá k.

Cách 1: mô phỏng xếp từng phòng

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int q;
    cin >> q;
    while (q--) {
        ll n, k, i;
        cin >> n >> k >> i;
        ll rooms = 0, rest = n;
        while (rest > 0) {
            rooms++;
            rest -= k;
        }
        ll room = 1, seat = i;
        while (seat > k) {
            room++;
            seat -= k;
        }
        cout << rooms << ' ' << room << ' ' << seat << '\n';
    }
    return 0;
}
```

Với $n = 10^9$ và $k = 1$, chương trình lặp 10^9 lần cho mỗi vòng `while` và mất **0,66 giây** cho một câu hỏi. Với $n = 9 \cdot 10^{18}$, $k = 2$, nó lặp $4{,}5 \cdot 10^{18}$ lần và không kết thúc.

Lê Thành Nam

10. Cách 2 - Công thức (n + k - 1)/k

Số phòng là ⌈n/k⌉, theo công thức làm tròn lên ở mục 3. Học sinh i (đánh số từ 1) đổi về đánh số từ 0 là i - 1, nên phòng (đánh số từ 0) là (i - 1)/k và chỗ (đánh số từ 0) là (i - 1) % k. Cộng 1 vào cả hai để quay về đánh số từ 1.

Cách 2: công thức làm tròn lên quen thuộc

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int q;
    cin >> q;
    while (q--) {
        ll n, k, i;
        cin >> n >> k >> i;
        ll rooms = (n + k - 1) / k;
        ll room = (i - 1) / k + 1;
        ll seat = (i - 1) % k + 1;
        cout << rooms << ' ' << room << ' ' << seat << '\n';
    }
    return 0;
}
```

Chương trình chạy tức thì và đúng ở ví dụ thứ nhất, nhưng ví dụ thứ hai in **0 1 1**. Với $n = k = 9 \cdot 10^{18}$, tổng $n + k - 1 \approx 1{,}8 \cdot 10^{19}$ vượt quá $9{,}22 \cdot 10^{18}$, phép cộng bị tràn số. Trong C++, tràn số với kiểu có dấu là **hành vi không xác định**: chương trình có thể in ra bất kì giá trị nào. Trên máy thử, nó in 0.

11. Cách 3 - Làm tròn lên không tràn số

Thay (n + k - 1)/k bằng n / k + (n % k != 0). Biểu thức `(n % k != 0)` có giá trị 1 khi còn dư (cần thêm một phòng cho các em còn lại) và 0 khi chia hết. Không có phép cộng nào giữa hai số lớn.

XEPPHONG.CPP: làm tròn lên an toàn

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("XEPPHONG.INP", "r")) {
        freopen("XEPPHONG.INP", "r", stdin);
        freopen("XEPPHONG.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        ll n, k, i;
        cin >> n >> k >> i;
        ll rooms = n / k + (n % k != 0);
        ll room = (i - 1) / k + 1;
        ll seat = (i - 1) % k + 1;
        cout << rooms << ' ' << room << ' ' << seat << '\n';
    }
    return 0;
}
```

Lê Thành Nam

XEPPHONG.CPP: làm tròn lên an toàn

```
23  }
```

■ ĐẾM SỐ PHÉP TÍNH

- Cách 1: O(n/k + i/k) mỗi câu hỏi. n = 10^9, k = 1: **0,66 giây** cho một câu; n = $9 \cdot 10^{18}$, k = 2: không kết thúc.
- Cách 2: O(1) nhưng **sai** khi n + k - 1 vượt phạm vi long long (in 0 thay vì 1 ở ví dụ thứ hai).
- Cách 3: O(1) và đúng với mọi dữ liệu: **0,003 giây** cho mỗi trường hợp trên.

■ BẪY THƯỜNG GẶP

- **Dùng `/` và `%` của C++ với số âm** khi cần làm tròn xuống hoặc số dư không âm. Hãy dùng floorDiv và modPos.
- **Quên đổi về đánh số từ 0:** viết i / k + 1 thay vì (i - 1)/k + 1 làm học sinh cuối mỗi phòng (i = k, 2k, ...) bị xếp sang phòng sau.
- **Tràn số khi làm tròn lên:** (a + b - 1)/b với a, b lớn. Tương tự, `x + k <= b` có thể tràn, hãy viết `x <= b - k`.
- **Đọc số tới 10^{18} vào biến int:** int chỉ chứa được tới khoảng $2{,}1 \cdot 10^9$. Mọi biến liên quan phải là long long.

Lê Thành Nam

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	PHÁT QUÀ	Chia có dư và làm tròn lên với số tới 10^{18}.
1.2	ĐẾM BỘI HAI SỐ	Số bội trong đoạn [a, b] và bao hàm - loại trừ với BCNN; cẩn thận số âm.
1.3	XẾP HÀNG	Đánh số theo hình "con rắn": chia lấy nguyên, chia lấy dư và tính chẵn lẻ của hàng.

Tự kiểm tra nhanh:

1. Tính thương và số dư (theo định lí chia có dư) của -100 chia cho 7, rồi cho biết C++ in ra gì với `-100 / 7` và `-100 % 7`.

(Đáp số: -100 = 7·(-15) + 5, thương -15, dư 5. C++ in -14 và -2.)

2. Có bao nhiêu số chia hết cho 6 trong đoạn [-20, 20]?

(Đáp số: ⌊20/6⌋ - ⌊-21/6⌋ = 3 - (-4) = 7, gồm -18, -12, -6, 0, 6, 12, 18.)

3. Bảng có 5 cột, ô đánh số từ 1 theo từng hàng. Ô số 15 và ô số 16 nằm ở hàng nào, cột nào (hàng, cột đánh số từ 1)?

(Đáp số: Ô 15: hàng (15 - 1)/5 + 1 = 3, cột (15 - 1) % 5 + 1 = 5. Ô 16: hàng 4, cột 1.)

BÀI 2 Đồng dư thức và phép toán modulo

Học xong bài này, em sẽ: